



ÉCOLE SUPÉRIEURE EN SCIENCES APPLIQUÉES – ESISA

Filière : Génie Logiciel & Systèmes Intelligents

Année Universitaire : 2025 / 2026

RAPPORT DE PROJET DE FIN D'ANNÉE (PFA)

MrcLens

Guide Touristique Intelligent & Culturel du Maroc
(Mobile & Web)

“Le Maroc à travers le temps, dans votre poche et sur votre écran.”

Réalisé par :

Zineb CHERRADI

Abdeljalil BOUARAIS (*Chef de projet*)

Maryem MABROUKI

Encadré par :

M. Othmane MEKOUAR

Mme Salma FENNANE

Fès, Juin 2026

Remerciements

Nous souhaitons débiter ce travail par l'expression de notre profonde gratitude envers toutes les personnes qui ont contribué, par leur soutien et leurs conseils, à la réussite de notre Projet de Fin d'Année.

Nos remerciements les plus sincères vont tout d'abord à **M. MEKOUAR Khalid**, président du jury, pour son accompagnement, sa bienveillance et la confiance qu'il nous a accordée tout au long de notre parcours. Son rôle a été déterminant dans la valorisation de notre travail.

Nous tenons également à remercier **M. MEKOUAR Youssef** et **M. Chihab Eddine**, membres du jury, pour l'attention portée à notre projet, ainsi que pour leurs remarques pertinentes et enrichissantes, qui nous ont permis d'améliorer significativement la qualité de notre réalisation.

Nous exprimons notre profonde reconnaissance à nos encadrants, **M. Othmane MEKOUAR** et **Mme Salma FENNANE**, pour leur accompagnement constant, leur disponibilité et leurs conseils judicieux. Leur encadrement rigoureux et bienveillant a été un pilier fondamental dans l'aboutissement de ce projet.

Enfin, nous adressons nos remerciements à **M. Iraqi Houssaini**, pour son soutien, sa disponibilité et l'intérêt qu'il a porté à notre projet. Son expérience et ses observations ont été très appréciées.

À tous, nous exprimons notre reconnaissance la plus profonde pour leur implication, leur temps et leur précieuse contribution.

TABLE DES FIGURES

3.1	Architecture hybride de la plateforme MrcLens	7
-----	---	---

LISTE DES TABLEAUX

1.1	Défis touristiques et impact sur le visiteur	2
1.2	Livrables attendus du projet MrcLens	3
3.1	Critères de sélection des technologies	10

TABLE DES MATIÈRES

Introduction Générale	V
1 Présentation du Projet	1
1.1 Contexte du Projet	2
1.2 Problématique	2
1.3 Périmètre du Projet	2
1.3.1 Périmètre Fonctionnel	2
1.3.2 Livrables Principaux	3
1.4 Méthodologie et Pipeline de Réalisation	3
1.5 Conclusion du Chapitre	4
2 Conception et modélisation	5
3 Architecture et Choix Technologique	6
3.1 Introduction	7
3.2 Architecture de la Plateforme	7
3.2.1 Écosystème Principal (Stack MERN)	7
3.2.2 Écosystème IA & Patrimoine	7
3.3 Framework et Technologies	8
3.3.1 Stack MERN - Application Principale	8
3.3.2 Module IA et Données Patrimoniales	8
3.3.3 Services et Outils Complémentaires	9
3.4 Justification des Choix Technologiques	10
3.5 Avantages de l'Architecture Hybride	11
3.6 Conclusion du Chapitre	11

INTRODUCTION GÉNÉRALE

Le Maroc est un pays riche en histoire, en culture et en paysages magnifiques. Chaque année, des millions de touristes viennent découvrir ses médinas, ses déserts et ses montagnes. Mais aujourd’hui, les voyageurs ont de nouveaux besoins : ils veulent des informations rapides, des conseils personnalisés et une expérience facile à organiser.

Face à ce constat, une question se pose : comment utiliser la technologie moderne pour améliorer l’expérience des touristes au Maroc ? Comment leur permettre de découvrir le pays plus simplement et plus intelligemment ?

C’est pour répondre à ces questions que nous avons créé **MrcLens**, une application web qui agit comme un guide touristique intelligent. Grâce à l’intelligence artificielle, MrcLens aide les visiteurs à découvrir le Maroc d’une nouvelle manière : en leur proposant des recommandations personnalisées, en facilitant les réservations d’hôtels et d’activités, et en leur offrant une assistance vocale et conversationnelle.

Pour réaliser ce projet, nous avons utilisé des technologies modernes et performantes : **React** pour l’interface utilisateur, **Node.js/Express** pour le serveur, **MongoDB Atlas** pour la base de données, **Cloudinary** pour gérer les images et vidéos, et **OpenAI GPT-5.5** pour l’assistant intelligent. L’application inclut également un système d’authentification sécurisé avec **JWT**.

Ce rapport présente notre travail de manière structurée. Nous commencerons par expliquer le contexte du projet et les besoins que nous avons identifiés. Ensuite, nous décrirons l’architecture technique et les choix que nous avons faits. Nous présenterons ensuite le développement de l’application, les fonctionnalités implémentées et les difficultés rencontrées. Enfin, nous terminerons par un bilan du projet et les améliorations possibles pour l’avenir.

À travers MrcLens, nous souhaitons montrer que la technologie peut vraiment améliorer l’expérience touristique et aider à faire découvrir le Maroc sous son meilleur jour.

CHAPITRE 1

PRÉSENTATION DU PROJET

Table des matières

1.1	Contexte du Projet	2
1.2	Problématique	2
1.3	Périmètre du Projet	2
1.3.1	Périmètre Fonctionnel	2
1.3.2	Livrables Principaux	3
1.4	Méthodologie et Pipeline de Réalisation	3
1.5	Conclusion du Chapitre	4

1.1 Contexte du Projet

Le tourisme représente un pilier essentiel de l'économie nationale, avec plus de 15 millions de visiteurs attendus en 2026. Pourtant, l'expérience des touristes dans les villes historiques comme Fès reste confrontée à plusieurs freins majeurs, comme illustré dans le tableau ci-dessous :

TABLE 1.1 – Défis touristiques et impact sur le visiteur

Défi	Impact sur le visiteur
Complexité de la médina	Problèmes d'orientation et risque d'égarement
Accès à l'information culturelle	Explications superficielles ou inexistantes sur les monuments
Barrière linguistique	Difficulté à communiquer avec les prestataires locaux
Manque d'interactivité	Expérience de visite passive et peu mémorable

Face à ces défis, il devient essentiel de repenser l'expérience touristique à travers des solutions numériques innovantes. C'est dans cette optique que s'inscrit notre projet MrcLens, qui vise à combiner technologie mobile, intelligence artificielle et valorisation du patrimoine culturel marocain pour offrir une expérience touristique améliorée.

1.2 Problématique

Cette problématique se décline en trois questions opérationnelles :

1. Comment rendre l'histoire et le patrimoine accessibles et engageants grâce à l'IA ?
2. Comment assister les touristes dans leurs besoins pratiques (logement, restauration, transport) sans surcharger l'interface principale ?
3. Comment concevoir une architecture technique scalable pour un déploiement progressif sur plusieurs villes ?

1.3 Périmètre du Projet

1.3.1 Périmètre Fonctionnel

Le projet couvre les fonctionnalités suivantes :

- Site web responsive React
- Module culturel : Avatar IA, analyse d'images architecturales
- Module vocal intelligent basé sur une API d'IA (GPT) permettant de proposer des recommandations de restaurants, riads et autres services à partir de commandes vocales.

TABLE 1.2 – Livrables attendus du projet MrcLens

Livrable	Format / Description
Application Web	URL déployée + code source React/Node.js
Backend & APIs	API Node.js / FastAPI + workflows n8n
Documentation API	Swagger / Postman collection + documentation des endpoints
Base de connaissances	Fichiers JSON (données historiques, lieux, services, etc.)
Base de données	MongoDB Atlas (schéma + collections + sauvegarde exportée)
Tests	Tests unitaires + tests API (Postman)
Architecture	Diagramme système (frontend, backend, IA, API externes)
Déploiement	Docker / Vercel / Render / Cloud setup
Documentation	Guide utilisateur + manuel technique + sécurité (PDF + Web)
Assets graphiques	Icônes, maquettes Figma, avatars vidéo (PNG/SVG/MP4)

1.3.2 Livrables Principaux

1.4 Méthodologie et Pipeline de Réalisation

Afin de garantir une exécution rigoureuse et itérative, le développement de MrcLens suit un pipeline structuré en quatre phases, regroupant les 12 étapes clés du projet :

Phase 1 : Préparation des données & Cartographie

1. Collecter les données historiques et images des monuments
2. Créer la base de données monuments + coordonnées GPS
3. Construire la carte interactive de la médina de Fès

Phase 2 : Développement des modules IA

4. Développer le module de détection par géolocalisation
5. Créer le système RAG (Retrieval-Augmented Generation) pour le guide virtuel
6. Créer et animer les personnages historiques (Avatar IA)

Phase 3 : Optimisation & Intégration

8. Convertir et optimiser le modèle pour mobile (TensorFlow Lite)
9. Implémenter la recommandation de parcours personnalisée
10. Intégrer les modules IA avec l'application mobile et le backend

Phase 4 : Tests & Amélioration Continue

11. Tester sur terrain réel (pilote Fès)
12. Améliorer les performances et l'UX avec les retours utilisateurs

1.5 Conclusion du Chapitre

Ce premier chapitre a posé les fondations du projet **MrcLens** en définissant le contexte touristique marocain, la problématique d'accompagnement numérique et la solution innovante proposée. Le périmètre fonctionnel, les livrables et le pipeline de réalisation ont été établis, traçant une feuille de route technique claire.

Au-delà du développement, MrcLens incarne une vision : allier intelligence artificielle et patrimoine culturel pour révolutionner l'expérience touristique au Maroc. Les bases sont désormais solides pour transformer cette ambition en réalité.

Le chapitre suivant entrera dans le concret avec l'analyse fonctionnelle détaillée et l'architecture du système.

CHAPITRE2

CONCEPTION ET MODÉLISATION

Table des matières

CHAPITRE3

ARCHITECTURE ET CHOIX TECHNOLOGIQUE

Table des matières

3.1	Introduction	7
3.2	Architecture de la Plateforme	7
3.2.1	Écosystème Principal (Stack MERN)	7
3.2.2	Écosystème IA & Patrimoine	7
3.3	Framework et Technologies	8
3.3.1	Stack MERN - Application Principale	8
3.3.2	Module IA et Données Patrimoniales	8
3.3.3	Services et Outils Complémentaires	9
3.4	Justification des Choix Technologiques	10
3.5	Avantages de l'Architecture Hybride	11
3.6	Conclusion du Chapitre	11

3.1 Introduction

Après avoir défini le contexte, la problématique et le périmètre fonctionnel du projet MrcLens dans les chapitres précédents, nous présentons désormais les fondations techniques de notre solution.

Pour répondre aux besoins spécifiques de notre plateforme touristique intelligente, nous avons adopté une **architecture hybride** combinant deux stacks technologiques complémentaires :

- Une stack **MERN** (MongoDB, Express, React, Node.js) pour l'application principale
- Un module **Python** avec **PostgreSQL** (pgAdmin 4) pour l'avatar IA et les données patrimoniales

Cette approche nous permet d'exploiter les forces de chaque technologie selon les besoins spécifiques de chaque fonctionnalité.

3.2 Architecture de la Plateforme

L'architecture de MrcLens se compose de deux écosystèmes distincts mais interconnectés :

3.2.1 Écosystème Principal (Stack MERN)

- **Frontend** : React.js (Web)
- **Backend** : Node.js et Express.js
- **Base de données** : MongoDB Atlas

3.2.2 Écosystème IA & Patrimoine

- **Langage** : Python pour la partie avatar intelligent
- **Base de données** : PostgreSQL géré via pgAdmin 4 pour le stockage des informations relationnelles sur les monuments

Cette architecture hybride est illustrée dans la figure ci-dessous :

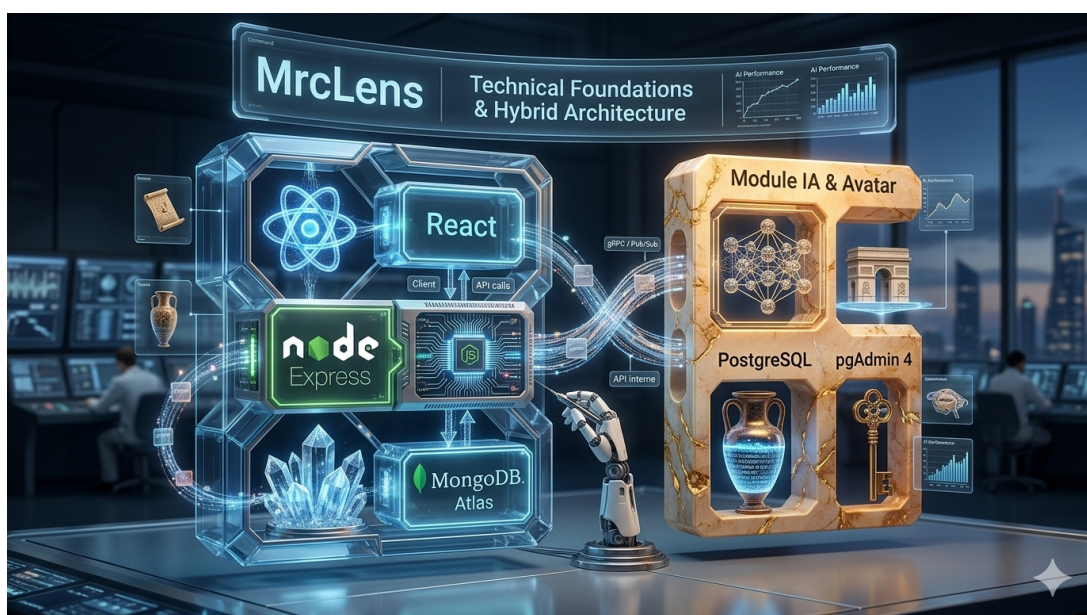


FIGURE 3.1 – Architecture hybride de la plateforme MrcLens

3.3 Framework et Technologies

3.3.1 *Stack MERN - Application Principale*

Frontend : React.js

Pour l'interface utilisateur, nous avons choisi l'écosystème React :

React.js pour l'application web :

- Bibliothèque JavaScript moderne et performante
- Architecture basée sur les composants réutilisables
- Virtual DOM pour des mises à jour optimisées
- Grande communauté et écosystème riche

Backend : Node.js et Express.js

Le serveur et l'API RESTful sont développés avec Node.js et Express :

- **Node.js** : Environnement d'exécution JavaScript côté serveur
 - Architecture asynchrone et non-bloquante
 - Même langage (JavaScript) pour frontend et backend
- **Express.js** : Framework web minimaliste et flexible
 - Gestion simplifiée des routes et middlewares
 - Création rapide d'APIs RESTful
 - Intégration facile avec MongoDB et l'authentification JWT

Base de Données : MongoDB Atlas

Pour la gestion des données de l'application (utilisateurs, réservations, préférences) :

- Base de données NoSQL orientée documents
- Structure flexible adaptée aux profils utilisateurs variés
- MongoDB Atlas : solution cloud gérée avec sauvegardes automatiques
- Scalabilité horizontale native
- Intégration native avec Node.js via Mongoose

3.3.2 *Module IA et Données Patrimoniales*

Python pour l'Intelligence Artificielle

Pour la fonctionnalité d'avatar intelligent qui raconte l'histoire des monuments, nous avons utilisé

Python :

- Langage de référence pour l'IA et le Machine Learning
- Bibliothèques riches pour le traitement du langage naturel (NLP)
- Intégration avec des APIs d'IA générative (OpenAI GPT-4o mini)

- Génération et animation d'avatars parlants
- Scripting efficace pour la narration automatique

PostgreSQL via pgAdmin 4

Pour le stockage des informations relatives aux monuments et au patrimoine culturel, nous avons choisi une base de données relationnelle :

PostgreSQL :

- Système de gestion de base de données relationnelle objet (SGBDRO)
- Robustesse et fiabilité pour les données structurées
- Respect de l'intégrité référentielle (clés étrangères, contraintes)
- Idéal pour les relations complexes entre monuments, villes, époques, etc.
- Support des requêtes SQL avancées et des jointures

pgAdmin 4 :

- Interface graphique d'administration de PostgreSQL
- Gestion visuelle des schémas de base de données
- Exécution et débogage des requêtes SQL
- Administration des utilisateurs et des permissions
- Outil de modélisation et de visualisation des données

3.3.3 Services et Outils Complémentaires

Cloudinary

Plateforme de gestion des médias dans le cloud :

- Stockage et optimisation des images de profil des utilisateurs de l'application
- Transformation d'images à la volée (redimensionnement, compression)
- CDN global pour un chargement rapide

Authentification et Sécurité

Pour garantir la sécurité des données utilisateurs et des échanges avec l'API, nous avons mis en place plusieurs mécanismes de protection robustes :

- **JWT (JSON Web Tokens) & Refresh Token :**
 - **Access Token** : Token de courte durée utilisé pour authentifier chaque requête envoyée à l'API.
 - **Refresh Token** : Token de plus longue durée permettant de renouveler silencieusement le token d'accès. Cela évite à l'utilisateur de devoir se reconnecter constamment, améliorant ainsi l'expérience utilisateur tout en maintenant une sécurité stricte.

- **Hachage des mots de passe (Hash / Bcrypt)** : Les mots de passe des utilisateurs ne sont jamais stockés en clair dans la base de données. Ils sont hachés (transformés en une chaîne de caractères irréversible) à l'aide de la bibliothèque **Bcrypt** avant d'être enregistrés dans MongoDB, garantissant ainsi la confidentialité des comptes en cas de fuite de données.
- **CORS (Cross-Origin Resource Sharing)** : Configuration stricte des en-têtes HTTP pour autoriser uniquement les domaines de notre frontend à communiquer avec notre backend Node.js.

Outils de Développement et Modélisation

- **Enterprise Architect** : Outil de modélisation UML (Unified Modeling Language) utilisé pour la conception, l'analyse et la documentation de l'architecture logicielle. Il nous a permis de réaliser les diagrammes de cas d'utilisation, de classes et de séquences pour structurer le projet avant le développement.
- **Visual Studio Code** : IDE principal avec extensions dédiées
- **Git & GitHub** : Versionning et collaboration
- **Postman** : Test et documentation des APIs
- **Swagger** : Documentation interactive des endpoints
- **Figma** : Design d'interface et prototypage
- **Docker** : Conteneurisation pour le déploiement

3.4 Justification des Choix Technologiques

Le tableau suivant résume les critères ayant guidé notre sélection :

TABLE 3.1 – Critères de sélection des technologies

Technologie	Justification
React	Développement d'interfaces web, composants réutilisables, performances optimisées côté client
Node.js / Express	JavaScript fullstack, asynchrone, rapide pour les APIs REST
MongoDB Atlas	Flexibilité NoSQL, cloud managé, adapté aux données utilisateurs
Python	Leader en IA/NLP, bibliothèques riches pour l'avatar intelligent
PostgreSQL (pgAdmin 4)	Données relationnelles structurées, intégrité des données patrimoniales
Cloudinary	Gestion média optimisée, CDN global, transformations automatiques
JWT	Authentification sécurisée, légère et standardisée
Enterprise Architect	Modélisation UML professionnelle, conception rigoureuse de l'architecture logicielle

3.5 Avantages de l'Architecture Hybride

Notre choix d'une architecture combinant MERN et Python/PostgreSQL présente plusieurs avantages :

- **Spécialisation** : Chaque technologie est utilisée pour ses points forts (NoSQL pour les utilisateurs, SQL pour les monuments, Python pour l'IA)
- **Performance** : MongoDB pour les lectures rapides, PostgreSQL pour les requêtes complexes sur les données culturelles
- **Maintenabilité** : Séparation claire des responsabilités entre l'application et le module IA
- **Scalabilité** : Chaque composant peut être mis à l'échelle indépendamment
- **Flexibilité** : Possibilité d'améliorer le module IA sans impacter l'application principale

3.6 Conclusion du Chapitre

Ce chapitre a présenté l'architecture technique complète de MrcLens, combinant la stack MERN pour l'application principale et Python avec PostgreSQL pour l'avatar intelligent et les données patrimoniales. Cette approche hybride nous permet d'exploiter le meilleur de chaque technologie : la flexibilité et la rapidité de développement du JavaScript moderne, couplées à la puissance de Python pour l'IA et la rigueur de PostgreSQL pour les données culturelles structurées.

Ces fondations techniques solides et bien pensées nous permettent désormais d'aborder sereinement la phase de développement. Le chapitre suivant détaillera l'implémentation concrète des fonctionnalités, les interfaces réalisées et les défis techniques surmontés pour donner vie à MrcLens.

BIBLIOGRAPHIE